

Project Status & Next Build Report

Report 2 | Current implementation, application connections, completion estimate and prioritized next steps

36%	~15%	0
Estimated roadmap complete	Production readiness estimate	Live PostgreSQL connection verified

Executive assessment

HPS has grown from a single-page storefront into a working browser prototype with a Node.js API and customer account flows. Catalog browsing, the Fabric.js Design Studio, server-calculated quotes, checkout order creation, authentication, design versioning, and saved addresses are implemented. PostgreSQL schema, migrations, seed tooling and repository paths exist, but no live database URL is configured, so PostgreSQL persistence has not been integration-tested. Payment processing, staff operations, production execution, object storage and deployment remain future work.

This is an engineering progress estimate, not a percentage of elapsed time or a claim of production readiness. The estimate gives partial credit for code that exists but has not passed its production acceptance gates.

Assessment date: 29 September 2026 | Repository: ofriday0420-code/hps | Branch: main

1. Project scope and progress method

The original 14-page implementation plan describes a production custom-print platform: responsive storefront, print design and quality checks, commerce, payments, accounts, administration, production, delivery, security, analytics and launch operations. This report evaluates the current repository against that end-to-end target.

Capability area	Weight	Build status	Weighted contribution
Brand, UX and storefront shell	12%	80%	9.6 points
Catalog and product discovery	8%	55%	4.4 points
Design Studio and account sync	12%	60%	7.2 points
Cart and checkout	12%	35%	4.2 points
Identity, designs and addresses	8%	60%	4.8 points
Backend and PostgreSQL	20%	18%	3.6 points
Payments, admin, production and delivery	18%	5%	0.9 points
Security, QA and deployment	10%	8%	0.8 points
Overall estimate	100%	35.5%	Rounded to approximately 36%

Comparison with Report 1: Report 1 estimated approximately 28%. This reassessment is approximately 36% after the PostgreSQL repository/migration/seed foundation and customer design/address slices were added. The earlier figure was a high-level estimate; this report makes the weighting explicit, so the change should be read as a directional update rather than precise earned-value accounting.

Important distinction: production readiness is only about 15%. Core flows still rely on local JSON/browser files, the live PostgreSQL path has not been exercised, and production payment, admin, print-file, operational and deployment capabilities are absent.

Status meanings

Label	Meaning
Implemented and smoke-tested	Feature exists and its local behavior was exercised; this does not imply production acceptance.
Implemented, database unverified	PostgreSQL code/schema exists, but no DATABASE_URL or reachable database was available.
Partial / prototype	User-facing or API foundation exists, but important end-to-end or production requirements are missing.
Not built	No working integration or operational workflow is present.

2. Current architecture and connection map

Layer	Current implementation	Connection / limitation
Customer browser	index.html; one HTML/CSS/JS file; Fabric.js 5.3.1 via CDN.	Opened directly as file:// in documented workflow; editor needs internet for CDN. Not a modular web app yet.
Browser-local state	localStorage for cart and per-product/front-back design drafts.	Survives same-browser refresh only; not an account or cross-device source of truth.
API server	Node.js built-in HTTP server; npm start; default 127.0.0.1:3000.	Browser requests API routes over HTTP. CORS currently allows origin null for local file:// use.
Catalog / price rules	data/products.json loaded on API start; quoteCart validates variants and calculates charges.	Still source for GET /api/products, server quotes and cart validation even when PostgreSQL configured.
Development persistence	data/users.json, data/orders.json; designs.json and addresses.json are local fallback stores.	Used when DATABASE_URL is absent. User/order writes use atomic temp-file rename; data files are not multi-process safe.
PostgreSQL connection	server/db.js loads dotenv, configures pg Pool from DATABASE_URL.	Connection timeout 5 seconds, max pool 10; production TLS certificate verification enabled. No URL configured; live connection unverified.
Database schema / tooling	database/001_initial_schema.sql, 002_auth_sessions.sql, server/migrate.js, server/seed.js.	Run migrations, then seed products/variants. Tables exist as SQL definitions until a database is configured and migrations run.
External services	Fabric.js CDN only.	No payment, courier, email/SMS, object-storage, analytics or deployment provider connected.

Request flow: Browser storefront/editor -> Node API (quotes, account, designs, addresses, orders) -> JSON fallback OR PostgreSQL repositories for selected domains. Quote calculation and catalog are currently backed by JSON, not PostgreSQL.

Database path now represented in code: API -> pg Pool -> DATABASE_URL -> PostgreSQL. Migration 001 builds commerce schema; migration 002 adds hashed persistent session records; seed command upserts categories, products and variants. Because connection setup is absent, the diagram is a configured-capable path, not a verified live connection.

3. What is implemented and how components connect

Area	Implemented behavior	Connection / verified boundary
Storefront	Responsive brand header, mobile navigation, product cards/details, search/category/price filters.	UI is served from the root index.html; catalog data includes 5 product types.
Design Studio	Fabric canvas, image upload, text/templates, duplicate/delete, color/opacity, front/back, safe-area and quality feedback.	Editor state first saves in browser; signed-in save/load can call account design endpoints. No print-ready export pipeline.
Cart and price quote	Persistent browser cart; quantity/removal; server quote verifies product, size, color, print sides, express and fees.	POST /api/cart/quote. Tested sample T-shirt total: BDT 550 (M + front print + delivery).
Authentication	Register/login/logout/current-user; scrypt + random salt + timing-safe check; HttpOnly session cookie.	PostgreSQL mode stores user and SHA-256 hash of random session token; JSON mode user file + process-memory sessions.
Orders	Checkout input validation, server-side repricing, public order ID; order item snapshots.	PostgreSQL mode transaction inserts order and items; JSON mode writes orders.json. No payment capture or idempotency.
Design account backup	Create/list/load designs; each save adds an immutable version; account ownership checked.	PostgreSQL designs/design_versions or JSON designs.json. UI keeps local save and syncs when signed in.
Saved addresses	Authenticated list/create/update/delete and checkout selection/save/update.	PostgreSQL addresses table or JSON addresses.json; routes scope reads/changes by user ID.
Database	18 core tables in initial schema and 7 indexes, constraints, UUID keys; hashed session migration.	Migration and seed scripts exist, but DB connection/schema application are not confirmed.

Current API surface

Route	Purpose / auth
GET /api/health	API status; reports PostgreSQL configured/connected or JSON mode. Current health returns HTTP 200 even if database check errors.
GET /api/products	Five products from data/products.json.
POST /api/cart/quote	Validates and recalculates quote server-side.
POST /api/orders	Validates checkout and creates guest/account order; current browser checkout does not explicitly send credentials.
POST /api/auth/register	Creates customer; unique-email conflict returns 409.
POST /api/auth/login	Verifies password, creates 7-day HttpOnly session.
POST /api/auth/logout; GET /api/auth/me	Session invalidation/current account.
GET /api/orders	Signed-in user order history.
GET/POST /api/designs; GET /api/designs/:id; POST /api/designs/:id/versions	Authenticated design library and immutable version saves.
GET/POST /api/addresses; PUT/DELETE /api/addresses/:id	Authenticated address book.

4. Data ownership and PostgreSQL connectivity detail

Data / domain	Current active source	PostgreSQL target status
Products, categories, variants	data/products.json	Seed writes categories/products/variants. API still reads JSON, not DB.
User credentials	users.json in JSON mode	users table via repository when DATABASE_URL set.
Sessions	Node process Map in JSON mode	auth_sessions stores token hash, user_id, expiry.
Orders and order lines	orders.json in JSON mode	orders + order_items in a transaction.
Saved designs / versions	designs.json + localStorage	designs + design_versions; versions immutable, current_version advanced.
Delivery addresses	addresses.json	addresses table with user ownership.
Cart	Browser localStorage	carts/cart_items schema exists; not yet used by API; no anonymous-to-user cart merge.
Payments, production, shipments	No live integration	Tables/schema only; no provider callbacks or business APIs/workflows.
Assets / print exports	Browser data URL/localStorage for preview/editor state	No private object storage or print-ready rendering service.

Database setup path

Step	Command / location	Result
Configure	Copy .env.example to .env; set DATABASE_URL privately.	No actual connection string is present in this workspace; do not expose credentials in chat or source.
Migrate	npm run db:migrate	Versioned SQL migration runner; expects a reachable PostgreSQL endpoint.
Seed	npm run db:seed	Upserts development catalog and all size/color variants. Run before order/design creation.
Run API	npm start	Pool is constructed on first DB use; API reports storage status from /api/health.
Verify	GET /api/health; then register/login, design/address/order smoke tests.	Must verify configured=true and connected=true and exercise every repository transaction against DB.

Connection status as of this report: No .env file / DATABASE_URL configured, API not permanently running, migrations and seed cannot connect. PostgreSQL readiness is code-prepared but live database connectivity and database-backed behavior are not verified.

5. Important gaps, risks and integration findings

Priority	Gap / observation	Why it matters / required correction
P0	No live PostgreSQL configuration or database integration test.	Database-backed account/design/address/order repositories could still expose schema or SQL/runtime mismatches; configure non-production DB and test migrations, seed, rollback and CRUD.
P0	Catalog/price source remains products.json even in PostgreSQL mode.	DB product edits/active flags/variant prices cannot govern quotes. Move reads and pricing to PostgreSQL repository while preserving API response contract and centralizing price rules.
P0	Browser checkout POST /api/orders does not specify credentials: include, unlike accountRequest.	Cross-origin file:// checkout can become a guest order despite a signed-in account. Serve same-origin in dev/prod or explicitly include credentials with a strict trusted-origin CORS policy; test user association.
P0	No live gateway verification, webhook signatures, idempotency key or payment reconciliation.	Online payment options are labels, not payment integrations; duplicate submissions/callbacks and false paid states are not protected.
P1	Cart schema is unused; browser localStorage is the only cart store.	No account persistence, cross-device cart or guest-to-account merge.
P1	Saved-design editor state includes inline image/data URL potential; no asset sanitization/rendering pipeline.	Payload ceiling is not safe asset management. Move binary assets into private object storage, validate/scan and render print-ready files server-side.
P1	JSON fallback uses file-based stores and process-local sessions.	Concurrency, multiple server instances, durability and session invalidation/restart behavior are not production-grade.
P1	API has no repository test suite in the project.	Only manual smoke tests have been run; add automated route, auth, ownership, pricing, transaction and database integration tests.
P1	Single-page index.html; no admin/front-end build pipeline.	Feature maintainability, separate staff authorization, deployment assets and robust browser testing are still missing.
P2	No courier, notifications, analytics, backups/restore, monitoring or deployment.	Operational lifecycle from order receipt through delivery and incident recovery is not yet complete.

The current browser flow and session cookie behavior need particular attention: account helper calls use credentials, but checkout order submission does not; the local file origin also makes cookies/CORS behavior dependent on browser rules. This should be fixed and tested while moving to a local served frontend before relying on account-linked checkout.

6. Next build roadmap: what, where and acceptance

Order	Build milestone	Primary location	Acceptance check
1	Connect a real non-production PostgreSQL instance.	Local .env (untracked), server/db.js, migration scripts.	/api/health confirms configured and connected; migrations apply twice safely; no credentials logged or committed.
2	Automated DB-backed tests for user/session/design/address/order repositories.	test/; package.json test scripts; optional test DB config.	CRUD, ownership isolation, uniqueness, version concurrency, order transaction rollback all pass against PostgreSQL.
3	Switch product/variant catalog and quote authority to PostgreSQL.	server/repositories.js or server/repositories/products.js; API catalog/quote routes.	Seed controls returned catalog; inactive/unknown variants rejected; server totals use DB-backed price data.
4	Cart persistence and checkout correctness.	Cart repository/routes; checkout frontend API request.	Auth cookie reaches order endpoint; guest cart merges after sign-in; idempotency prevents duplicate order on retries.
5	Payment adapter + verified sandbox webhook for one Bangladesh provider.	server/payments/, webhook route, payment repository.	Signature verification, replay/idempotency protection, reconciliation, success/failure/refund test cases.
6	Admin/authorization and design review.	apps/admin/ (or separate admin surface), server auth/authorization, routes.	Customer cannot access staff APIs; staff actions audited; approve/reject reason and revision flow work.
7	Production file/assets and notifications.	Private object storage adapter, design-export service, notification jobs.	Validated uploads; immutable order design reference; reproducible print export; order-state notifications.
8	Production deployment and operations.	Deployment/IaC, CI, monitoring, backups/runbooks.	HTTPS, managed secrets, health/readiness, alerting, backup restore and rollback tested.

Recommended immediate next action: configure a private development PostgreSQL DATABASE_URL and run migrations plus seed. Do not add live customer or payment data. After connection is confirmed, test the DB-backed accounts/designs/addresses/orders, then migrate product reads and quote calculations so PostgreSQL becomes the actual source of truth.

7. Project files, run instructions and conclusion

File / folder	Role
index.html	Current storefront/editor/cart/checkout/account integration; still monolithic.
server/index.js	Node HTTP server, API routes, validation and JSON-vs-PostgreSQL switching.
server/db.js	dotenv and PostgreSQL pool, health check, shutdown.
server/repositories.js	PostgreSQL queries/transactions for users, sessions, orders, designs, addresses.
server/migrate.js; database/*.sql	Versioned SQL migrations and migration history.
server/seed.js; data/products.json	Upsert the development product catalog and variants; source catalog currently JSON.
data/*.json	Development fallback data stores; no production durability/concurrency guarantee.
package.json	npm start/dev, db:migrate, db:seed; pg and dotenv dependencies.
README.md; database/README.md	Developer startup, routes and DB setup notes.
HPS_Custom_Print_Design_Final_Project_Implementation_Plan.pdf	Original 14-page target scope and architecture reference.
HPS_Custom_Print_Design_Project_Status_and_Next_Build_Report.pdf	Report 1 snapshot; prior estimate approximately 28%.

Run locally (JSON mode)

npm start

GET http://127.0.0.1:3000/api/health

No DATABASE_URL means JSON development mode. Configure DATABASE_URL privately, then run npm run db:migrate, npm run db:seed, and npm start to attempt PostgreSQL mode.

Overall conclusion

The customer-facing foundation and several account-level capabilities are implemented and manually exercised. HPS is not yet a production commerce platform: its PostgreSQL path is unverified, database products do not yet drive catalog/quote calculation, and payment, administration, production, delivery integrations, automated quality gates and deployment are incomplete. The project estimate is approximately **36% of the planned roadmap**, with approximately **15% production readiness**. The next progress gate is a verified PostgreSQL connection and repeatable database integration tests.

Document prepared from the current repository, its documentation, the existing Report 1 PDF and the original implementation-plan PDF. No database credentials are included.